SentinelPay Project Structure and Code Guide

Overview

SentinelPay is a server-rendered Django project. Business responsibilities are divided into Django apps so that HTTP routing, form validation, payment orchestration, fraud evaluation, audit records, and presentation do not all live in one module.

Repository tree

```
sentinelpay/
??? audit/
?   ??? migrations/
?   ??? models.py
?   ??? services.py
??? banking/
?   ??? management/commands/seed_demo.py
?   ??? migrations/
?   ??? context_processors.py
?   ??? forms.py
?   ??? models.py
?   ??? permissions.py
?   ??? views.py
??? fraud_engine/
?   ??? engine.py
?   ??? tests/
??? payments/
?   ??? migrations/
?   ??? tests/
?   ??? admin.py
?   ??? forms.py
?   ??? gateway.py
?   ??? models.py
?   ??? services.py
?   ??? states.py
?   ??? urls.py
?   ??? views.py
??? sentinelpay/
?   ??? settings.py
?   ??? urls.py
?   ??? wsgi.py
??? static/css/site.css
??? templates/
?   ??? banking/
?   ?   ??? customer_profile.html
?   ??? payments/
?   ??? registration/
?   ??? base.html
??? docs/
??? conftest.py
??? manage.py
??? pytest.ini
??? requirements.txt
??? README.md
```

The distribution ZIP intentionally excludes `db.sqlite3`, Git metadata, virtual environments, Python bytecode, and pytest cache data. `.env.example` contains placeholders and is included for setup reference; a real `.env` file is not included.

Project-level files

Path	Purpose
`manage.py`	Django command-line entry point for checks, migrations, seeding, and the development server.
`sentinelpay/settings.py`	Installed apps, middleware, templates, static files, authentication, environment variables, and database selection. SQLite is the default; optional MySQL requires environment configuration.
`sentinelpay/urls.py`	Root route table: admin, login, logout, overview, and the payments URL include.
`sentinelpay/wsgi.py`	WSGI entry point for compatible application servers.
`requirements.txt`	Pinned Django and pytest dependencies; MySQL client is optional.
`pytest.ini`, `conftest.py`	Pytest/Django settings and shared customer, account, and beneficiary fixtures.
`.env.example`	Placeholder configuration examples; it is not automatically loaded by Django.
`.gitignore`	Excludes `.env`, SQLite database, virtual environments, Python cache files, and pytest cache.
`README.md`	Project overview, demonstration rules, setup steps, test commands, and limitations.

Django applications and why their code is separated

`banking/`

Models the financial data used by the sample. `Customer` links one-to-one with a Django user and stores user-provided profile fields plus explicit KYC/risk placeholders. `BankAccount` stores a synthetic account number, type, status, currency, available balance, and ledger balance. `Beneficiary` can be customer-owned or project-wide, and `Transaction` records each payment's account, optional recipient, simulator status, risk score, and timestamps.

- `forms.py`: customer profile form validates date of birth and normalizes Indian mobile numbers. KYC and risk values are intentionally excluded from customer-editable fields.
- `models.py`: shared banking entities used by payment validation and the dashboard.
- `views.py`: customer overview and profile editing. Customer accounts/payments are scoped to the authenticated user; the profile view updates only that user's profile and corresponding Django user name/email.
- `permissions.py`: single permission rule for fraud reviewers (staff, superuser, or membership in `Fraud Analysts`). Keeping this logic centralized makes nav visibility and view authorization consistent.
- `context_processors.py`: exposes the reviewer permission to templates for conditional navigation. This is for display only; protected views check permission on the server too.
- `management/commands/seed_demo.py`: creates synthetic demo users, customer profiles, savings accounts, and a shared sample beneficiary using an environment-supplied password. It does not reset existing records.
- `migrations/`: database schema history. The profile/transaction migration backfills profile names/emails from existing Django users and one transaction per existing payment; account ledger balance begins at the existing balance as a cutover snapshot, not a reconstruction of history. Review and apply migrations deliberately; do not delete or replace migrations to repair a database.

`payments/`

Owns the payment workflow and its records. This app is separated from `banking/` because payment validation and state transitions are business processes that use, but do not own, bank account data.

- `models.py`: `Payment` stores customer, source account, optional beneficiary, amount, status, idempotency fingerprint, fraud evidence, simulator code, and timestamps. `LedgerEntry` records a posted debit, with uniqueness constraints to prevent duplicate posting. `FraudCase` tracks review work.
- `forms.py`: validates customer input and querysets. The payment form only exposes the current customer's accounts, and only their own or project-wide beneficiaries. Filter forms validate transaction search and monitoring filters.

- `views.py`: translates HTTP requests into page responses. It manages the payment draft in the session, handles review/confirmation, scopes receipts and statements, and gates fraud operations.
- `services.py`: payment orchestration called by the view. It verifies account/beneficiary ownership and activity, enforces idempotency, evaluates fraud, saves audit stages, calls the simulator, creates a linked banking transaction from the persisted outcome, makes the ledger debit on success, and creates cases for held/rejected payments.
- `states.py`: the allowed status transitions. A single function validates and saves each transition to prevent arbitrary status changes.
- `gateway.py`: deterministic simulator adapter. Merchant codes choose success, decline, timeout, or error responses; no external payment network is contacted.
- `urls.py`: payment route names and URL patterns, kept out of the project-wide URL file for clearer app boundaries.
- `admin.py`: exposes payment and ledger records through Django admin.
- `tests/`: checks fraud-related payment behavior, idempotency, ownership, ledger constraints, access control, CSV scope, and request flows.
- `migrations/`: versioned schema changes for payment and fraud-case tables.

`fraud_engine/`

Contains the standalone, deterministic rule engine. `engine.py` accepts a typed `PaymentContext`, evaluates rule functions, sums/caps rule points, and returns a score, band, decision, and explanations. The payment service builds the context from persisted server-side data and saves the result. The engine does not accept a score from the browser and is not a machine-learning model or calibrated probability.

`tests/` tests rule triggers, score combinations, and decision thresholds without coupling rule behavior to the web interface.

`audit/`

Provides a shared append-only record of significant actions.

- `models.py`: `AuditEvent` stores actor, action, object and correlation references, metadata, and time. Existing rows cannot be updated through the model's `save()` implementation.
- `services.py`: `record_event()` gives payment orchestration one reusable API to append stage and outcome events.
- `migrations/`: audit table schema history.

Audit records support payment timelines and case history; they are not currently exposed as an independent audit-history page.

`sentinelpay/`

Contains Django configuration rather than product domain logic. `settings.py` selects database configuration and template/static handling. `urls.py` declares global routing and uses Django's standard login, logout, and admin views.

`templates/` **and** `static/`

- `templates/base.html`: shared document shell, branding, accessible navigation, messages, and footer.
- `templates/registration/login.html`: sign-in form presentation; authentication itself is Django's built-in view.
- `templates/banking/dashboard.html`: customer details, account balances, spend, activity, and recent-transaction presentation using context from the banking view.
- `templates/banking/customer_profile.html`: customer-editable name and contact/profile fields; it does not expose KYC/risk status as editable inputs.
- `templates/payments/`: payment form, confirmation/review, transaction list, receipt, fraud queue, and case detail.
- `static/css/site.css`: shared responsive styles, light banking surfaces, visual indicators, and CSS animations. Reduced-motion preferences disable decorative motion.

Presentation code displays backend values. It does not independently create payment results, risk scores, ledger entries, or fraud alerts.

Design rationale

1. **Keep HTTP behavior separate from domain operations.** Views coordinate forms, sessions, and responses; the payment service owns the payment process.

2. **Enforce trust boundaries on the server.** Forms, services, queryset scoping, permission checks, state transitions, and database constraints are authoritative. Hiding a link or animating an indicator is not access control.
3. **Keep fraud rules independently testable.** Pure rule functions accept a typed context and can be tested without browser requests or database setup.
4. **Record facts rather than UI states.** Payments, audit events, fraud cases, and ledger entries persist the outcomes that pages display.
5. **Use a local deterministic adapter.** The gateway interface demonstrates outcomes in tests and development without implying a real provider integration.
6. **Keep schema evolution explicit.** Django migrations describe database changes, enabling the app to initialize a new database or incrementally update an existing one.

Local development and verification

From the project directory in Windows PowerShell:

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
$env:DJANGO_SECRET_KEY = "replace-with-a-local-development-secret"
$env:DEMO_PASSWORD = "choose-a-12+-character-password"
python manage.py migrate
python manage.py check
python -m pytest -q
python manage.py runserver
```

The sample project runs against SQLite by default. Do not copy a database with real or private records into a source archive. The MySQL configuration is optional and unverified.